

Here's a curated list of **production-level Kubernetes (K8s) interview questions** focused specifically on **namespaces**, along with **detailed answers**. These questions are aimed at intermediate to advanced roles like **DevOps Engineers, SREs, or Platform Engineers**.

Kubernetes Namespaces: Interview Questions & Answers (Production-Level)

1. What is a namespace in Kubernetes and why is it used in production?

Answer:

A namespace is a logical partition within a Kubernetes cluster. In production, namespaces are used to:

- **Isolate environments** (e.g., dev, staging, prod)
 - **Segment teams or services**
 - Apply **RBAC**, **resource quotas**, and **network policies**
 - Improve **multi-tenancy** and **operational governance**
-

2. How does Kubernetes enforce resource isolation between namespaces?

Answer:

Resource isolation is achieved using:

- **ResourceQuotas**: Limit CPU, memory, storage, and object counts per namespace.
- **LimitRanges**: Set default and max/min values for container resources within a namespace.
- **NetworkPolicies**: Restrict or allow traffic between pods in different namespaces.

3. Can two pods in different namespaces have the same name? Explain.

Answer:

Yes. Pod names only need to be unique **within the same namespace**. This allows multiple applications or environments to use the same naming convention independently.

4. How do you switch between namespaces without changing the default context?

Answer:

Use the `-n` or `--namespace` flag with your kubectl command:

```
kubectl get pods -n <namespace>
```

This allows temporary use of a namespace without modifying kubeconfig.

5. What are the default namespaces in Kubernetes and their purpose?

Answer:

Namespace	Purpose
default	Default namespace for user resources when none is specified
kube-system	Contains system components (e.g., kube-dns, kube-proxy)
kube-public	Globally readable; used for cluster-wide public data
kube-node-lease	Stores Lease objects used for node heartbeats

6. How do you apply RBAC policies on a per-namespace basis?

Answer:

Create **Roles** and **RoleBindings** instead of ClusterRoles:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

And bind it:

```
kind: RoleBinding
...
subjects:
- kind: User
  name: dev-user
...
```

7. What are some best practices for namespace usage in production?

Answer:

- Use **separate namespaces per environment** (dev/staging/prod).
- Assign **resource quotas and limits** to each namespace.
- Enforce **RBAC and Network Policies**.
- Use **naming conventions** (e.g., **team-env**, **app-role**).
- Avoid deploying apps in **kube-system**.

8. How do you clean up all resources in a namespace but keep the namespace?

Answer:

```
kubectl delete all --all -n <namespace>
```

To also remove configmaps, secrets, PVCs, etc., run:

```
kubectl delete $(kubectl api-resources --verbs=list --namespaced -o name | xargs -n1 -I {}  
kubectl get {} --no-headers -n <namespace> -o name)
```

9. How can you monitor namespace usage and enforce limits in production?

Answer:

- Use **ResourceQuota** and **LimitRange**.
- Monitor usage via Prometheus/Grafana or [kubectl describe quota](#).
- Alert on usage nearing quota limits to prevent service disruption.

10. Can you access resources across namespaces? How?

Answer:

Yes, with proper configuration:

- Pods can access Services in other namespaces via [<service>.<namespace>.svc.cluster.local](#)
- RBAC must allow cross-namespace access.

- Network policies must not block traffic.

Example:

```
curl http://web-service.dev.svc.cluster.local
```

11. How would you handle multi-tenancy in Kubernetes using namespaces?

Answer:

- Create **isolated namespaces** per tenant.
 - Use **network policies** to restrict inter-tenant communication.
 - Apply **resource quotas** to prevent noisy neighbors.
 - Use **RBAC** to limit access.
 - Optionally, integrate with **OPA/Gatekeeper** or **Kyverno** for governance.
-

12. What happens when you delete a namespace?

Answer:

- All resources within the namespace are **deleted recursively**.
 - This includes pods, services, secrets, configmaps, PVCs, etc.
 - The process may be delayed if any resource has a **finalizer**.
-

13. Can you use the same namespace name after deletion?

Answer:

Yes, but only **after Kubernetes finishes garbage collecting** all resources and removes the namespace object fully (can take a few seconds to minutes). Use `kubectl get ns <name>` to check deletion status.

14. How do you debug stuck namespaces (e.g., “Terminating” forever)?

Answer:

Identify resources with finalizers using:

```
kubectl get <resource> -n <namespace> -o json | jq '.metadata.finalizers'
```

- - Remove finalizers manually using `kubectl patch` or `kubectl edit`.
 - Use tools like `kubectl proxy` or API calls to force-remove.
-

15. How do you enforce a namespace creation policy?

Answer:

Use:

- Admission controllers like `NamespaceLifecycle`
 - OPA/Gatekeeper or Kyverno policies
 - GitOps tools (e.g., ArgoCD) to automate and validate namespace definitions
-